secrets. You can enable encryption by following the instructions in https://kubernetes.io/docs/tasks/administer-cluster/encrypt-data/.

> **CAUTION** The official Kubernetes documentation lists `aescbc` as the strongest encryption method supported. This is an unauthenticated encryption mode and potentially vulnerable to padding oracle attacks as you'll recall from chapter 6. You should use the `kms` encryption option if you can, because all modes other than `kms` store the encryption key alongside the encrypted data, providing only limited security. This was one of the findings of the Kubernetes security audit conducted in 2019 (https://github.com/trailofbits/audit-kubernetes).

- Anybody with the ability to create a pod in a namespace can use that to read the contents of any secrets defined in that namespace. System administrators with root access to nodes can retrieve all secrets from the Kubernetes API.
- Secrets on disk may be vulnerable to exposure through *path traversal* or *file exposure* vulnerabilities. For example, Ruby on Rails had a recent vulnerability in its template system that allowed a remote attacker to view the contents of any file by sending specially crafted HTTP headers (https://nvd.nist.gov/vuln/detail/CVE-2019-5418).

> **DEFINITION** A ***file exposure vulnerability*** occurs when an attacker can trick a server into revealing the contents of files on disk that should not be accessible externally. A ***path traversal vulnerability*** occurs